01 演算法在計算中的角色

Part I Foundations

本章定位

先把合法輸入、可接受輸出與終止條件寫成契約，再依相同成本模型比較演算法、資料表示與硬體；本章用重複鍵排序、路徑/巡迴及交會點數例，區分一般正確性、隨機保證與有限測試。

前置知識

基本函數、集合、程式執行與離散輸入表示。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 把問題、輸入集合、輸出關係與演算法規格分開。
- 用正確性、終止性與資源成長描述演算法品質。
- 區分抽象效率與特定硬體上的常數成本。
- 1.1 Algorithms
- 1.2 Algorithms as a technology

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

1.1 Algorithms

computational problem先定合法輸入集合D，以及每個x∈D可接受的輸出集合R(x)。例如同一組鍵可有多個合法排序結果，故規格通常是關係，不必是唯一公式。instance是其中一個具體x；algorithm是把輸入轉為輸出的明確可執行程序。

確定性演算法A的total correctness要求對**每個**x∈D均終止，且結果A(x)∈R(x)。partial correctness只要求「若終止，結果合法」；還須另證終止，才得到total correctness。合法輸入之外是拒絕、未定義或另一種結果，也必須先約定。

有限程式文字可以描述無限迴圈；「只有有限行偽碼」不證明每次執行都有限。測試若只核若干x，也不涵蓋規格中的全稱量詞。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

排序同時保有序與每次出現

輸入A=[(3,a),(1,b),(3,c)]，每筆為(key,identity)；keys具total order且比較成本另有明定。合法輸出必同時滿足①keys非下降；②與輸入是同一批records的permutation，不遺失、不複製。[(1,b),(3,a),(3,c)]符合兩者；[(1,b),(3,a)]有序卻遺失c，不是排序答案。

若再要求stability，相等key在輸出中保留輸入先後，所以a須先於c。非穩定的[(1,b),(3,c),(3,a)]仍可符合前兩項，但不符穩定排序規格。空陣列的結果是空陣列，兩項條件都成立。

若key可能含沒有一般total-order語義的值，必須先定比較政策；不能默默把無法比較的值當成已滿足排序條件。record identity讓重複鍵的保存與先後可以直接檢查。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

一個可完整證明的演算法

規格：A長n≥0，找值v；輸出最小i使A[i]=v，若不存在回NIL。索引從1開始，邏輯and採由左到右short-circuit。

```
FIRST-MATCH(A, v)
    n = A.length
    i = 1
    while i <= n and A[i] != v
        i = i + 1
    if i <= n
        return i
    return NIL
```

while開始前，A[1..i−1]沒有v。初始空前綴成立；每步確認A[i]≠v才使i增加，因此維持。離開時若i≤n，該處等v且前面皆不等，正是最小索引；否則i=n+1，全陣列皆無v。variant n−i+1每次迭代減1、迴圈內非負，故至多n次迭代，證明終止。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

成功、失敗與空輸入的完整 trace

A=[4,9,4]，v=4：i=1檢查1≤3、A[1]=4，while直接停止，回1。第二個4位於3，不影響「最小索引」。

同一A找v=7：

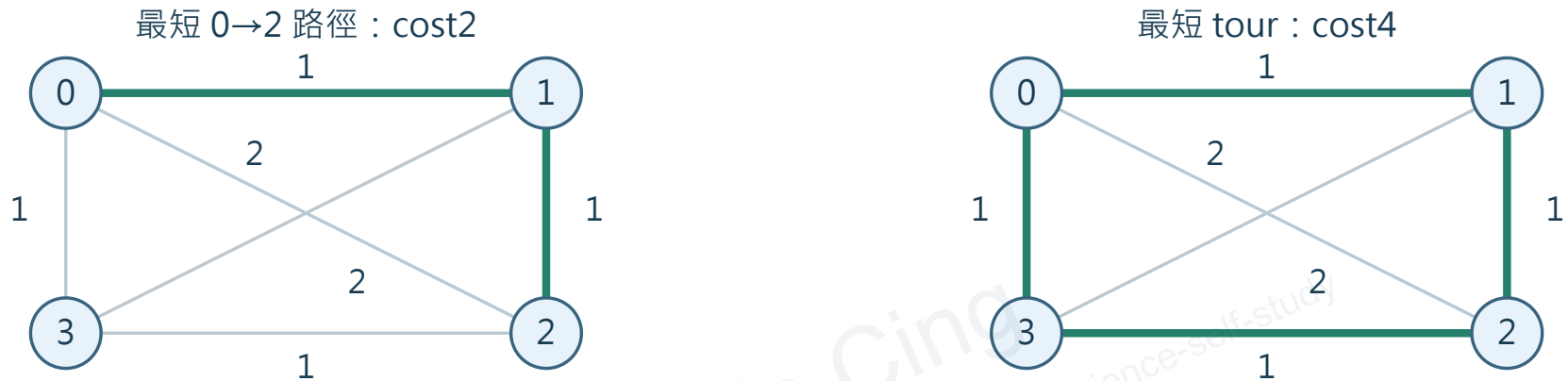
檢查前i	i≤n	鍵比較	後續狀態
1	真	4≠7	i=2
2	真	9≠7	i=3
3	真	4≠7	i=4
4	假	不讀A[4]	回NIL

n=0時第一次i≤n已假，也不讀A[1]。有界索引與short-circuit條件是偽碼可實作的一部分，不能省略後再用測試碰運氣。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

問題相似，合法答案卻不同

相似輸入，輸出契約不同



兩圖是同一個無向完整圖；對角線交點不是頂點。路徑不用訪全部點，tour須訪全點再返回。

無向完整圖的外圈四邊權1、兩條對角線權2。從0到2的最短路徑cost2，可走0→2或0→1→2，不要求拜訪3。TSP optimization要求訪問所有點恰一次再回起點，最小tour是0→1→2→3→0，cost4。

固定起點0、把反向視為同一tour，四點完整無向圖只有三種：01230成本4、01320成本6、02130成本6。這個有限枚舉證本**四點instance**的最優值；不是一般TSP的多項式算法。

表示法決定可行性與搜尋空間

LCS要求順序不變的subsequence，可刪位置而不必連續。X=ABA、Y=BAA的AA與BA都是長2共同subsequence；長3只能各取整串，兩串不同，所以最長長度2。共同substring則另要求位置連續，是另一問題。

X的8種位置選擇對應ε、A、B、A、AB、AA、BA、ABA；重複字元使不同位置子集得到相同字串，故2^n數的是位置選法，不保證2^n種不同字串。

依賴圖a→c、b→c、c→d要求先有a/b再做c、最後d；a,b,c,d與b,a,c,d合法，a,c,b,d不合法。若再加d→b，就要求b<c<d<b而矛盾，沒有topological order。選擇graph而非單一list表示依賴，才能清楚檢查這種無解情況。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

困難性與近似的量詞

decision problem只問yes/no；optimization problem找最佳值/解。TSP decision問「是否存在tour成本≤B」，optimization找最小成本tour。NP-complete通常用在decision版，optimization版談NP-hard，不能省略問題版本。

NP-complete的歸約性質表示，若其中一個decision problem有多項式算法，所有NP problems都有。單憑NP-complete標籤不構成「已證不存在多項式算法」；還要有相應複雜度分離前提。

approximation要有明確可行解與目標界，如minimization中cost(output)≤ρ・OPT；不能由「答案看起來好」推出此保證。heuristic、exact exponential algorithm、特殊輸入類別與approximation解不同需求，其時間/正確性前提應分開。

Lin Cing
https://github.com/xinhi6716/computer-scien

隨機答案要說明對誰取機率

固定合法輸入x，隨機bits為r。Monte Carlo保證可以是Pr_r[A(x;r)正確]≥2/3且時間有界；機率取自內部亂數，不是「隨機挑一些輸入測試有2/3通過」。每個x的保證不能由未定義的平均輸入成功率代替。

對相同binary decision獨立執行3次、多數表決，錯誤機率至多$C(3,2)(1/3)^2(2/3)+(1/3)^3=7/27$，正確至少20/27。獨立性不可省；若三次共用同一隨機選擇，可能一起錯而毫無改善。一般search輸出若有多個不同正確解，也不能直接做字串多數表決。

零錯誤隨機算法與終止/期望時間是另一組規格。有限期望時間可推出幾乎必然終止，但不表示每條無限亂數序列都在統一有限步內結束；這與確定性total correctness的量詞不同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

資料結構與平行性各改變哪個成本

找任意元素的array可能要掃n項；已排序array可用binary search，但插入可能搬Θ(n)項；hash table的期望時間依hash/負載模型而定。沒有一種資料結構同時無條件把所有操作變成常數。

平行工作例：4個彼此獨立unit tasks全部完成後，再接4個unit tasks的串行chain。work W=8、span S=5。1、2、4個processors的完成時間分別8、6、5；每個P下$T_P \geq \max(W/P, S)$，speedup $W/T_P \leq W/S = 1.6$。

W/S是加速比上界，不是「processor超過1.6就再無收益」。本例P=2已超過1.6，增加到4仍由6降5。排程可達的時間還取決於DAG與整數processor數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

1.2 Algorithms as a technology

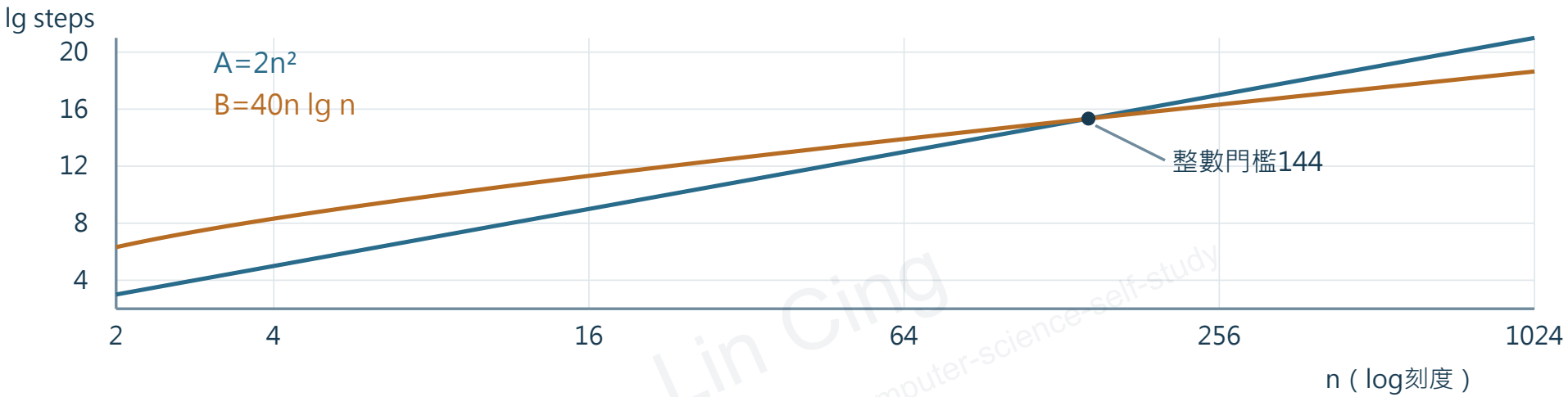
算法與硬體共同決定時間。若算法所需操作數為I(n)、機器對這些操作的有效處理率為R，簡化模型T(n)=I(n)/R。比較時要讓n、操作定義、R與單位一致；memory、I/O與同步可使單一R模型失效。

令$I_A(n)=2n^2$、$I_B(n)=40n \lg n$，先限定$n \geq 2$、同一種抽象step與相同R。A常數較小但二次成長，B是n log n。這兩個是明確模型，不是聲稱任何insertion/merge sort實作的固定精確指令數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

交會點必須核對兩側

同一 step 模型：n≥2；曲線只比較公式，不是實測



B<A等價n−20lg n>0。g(n)=n−20lg n，其導數1−20/(n ln2)，n>20/ln2後嚴格遞增。g(143)<0、g(144)>0，因此n≥2的整數範圍內，B從n=144起持續勝出；小區間也由先降後升的形狀排除漏解。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

交會點必須核對兩側 （續）

n	A steps	B steps	較少step
16	512	2560	A
128	32768	35840	A
256	131072	81920	B
1024	2097152	409600	B

圖使用log座標呈現同一模型。曲線相交附近的連續n不等於整數題目的門檻，strict inequality也須核對是否恰相等。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

更快硬體仍要除回同一時間單位

改令A機器$R_A=10^9$steps/s，B機器$R_B=10^7$steps/s，A原始吞吐快100倍；算法仍用上一頁兩式。$n=10^6$時A需2×10^{12}steps，時間2000s；B需約7.9726274×10^8steps，時間79.7263s。B雖處理率較慢，總時間仍約快25.086倍。

$n=1000$時A為.002s，B約.0398631s，反而A較快。此模型的時間交會改為$n-2000\lg n>0$，不能繼續套相同R時的144門檻。

大O只給上界集合，不能直接比較兩個帶未知常數的式子哪個實作較快。固定常數下$n/\log n$最終趨無窮，才支持「足夠大時較低成長階可抵銷常數劣勢」的趨勢。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

輸入大小與可用記憶體也是技術條件

整數N以binary表示的長度L=⌊lg N⌋+1。迴圈執行N次對數值是Θ(N)，對輸入bits卻可達Θ(2ᴸ)，不能稱其為一般bit-size的線性時間。array長n若每key是w-bit word，輸入占Θ(nw) bits，單次比較是否O(1)依word/cost模型。

若固定記憶體只容納M筆，n>M時原本的in-memory模型必改，外部I/O與buffer策略可能主導。平行/快取/網路不是可任意忽略的小常數，分析應先選問題規模與成本邊界。

算法會出現在application、compiler、runtime、OS、network與hardware設計各層。所需的是能把新問題定成契約、選合適表示法並證明正確/成本，而不是只記一個已知程式碼名稱。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 01。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study